

A PROJECT ON

HOSPITAL MANAGEMENT SYSTEM

SUBMITTED BY:

Utkarsh Pawade

UNDER THE GUIDANCE OF:

Prof. Neeti Chandrakar

Faculty Member

ACKNOWLEDGEMENT

A project usually falls short of its expectation unless aided and guided by the right persons at the right time. I avail this opportunity to express my deep sense of gratitude towards Mr. Nitin Kudale (Center Coordinator) and Mr. Yogesh Kolhe (Course Coordinator).

I am deeply indebted and grateful to them for their guidance, encouragement, and deep concern for my project. Without their critical evaluation and suggestions at every stage of the project, this project could never have reached its present form.

Last but not least, I thank the entire faculty and staff members for their continuous support.

Utkarsh Pawade

1. INTRODUCTION TO PROJECT

The web-based "Hospital Management System" (HMS) project is an attempt to simulate and modernize the core operational concepts of hospital patient management, doctor scheduling, consultation bookings, digital prescriptions, pharmacy inventory management, and transaction ledgers. The system enables patients to discover qualified medical specialists, search for doctors across various clinical departments on specified dates, choose a convenient time slot, and make online reservations for consultation appointments.

The system provides a Quick Search facility that allows visitors to browse details about available doctors, department specializations, consultation fees, and hospital operational hours without needing to log in. However, if a patient wishes to book a consultation slot, view digital medical prescriptions, or complete online payments, logging into their personal account is mandatory.

The system allows patients to search for doctors available across various hospital departments, namely "Cardiology", "Neurology", "Pediatrics", "Orthopedics", and "Dermatology" for a particular date. The system displays all relevant doctor details such as Doctor ID, Doctor Name, Specialization, Room Number, Consultation Fee, and Availability Status.

Here we provide a quick search facility which displays a list of available doctors and allows the patient to choose a specific doctor and time slot (in 30-minute intervals between 09:00 AM and 05:00 PM). Then the system checks for slot availability. If the slot is available, the system permits the patient to proceed with booking. Otherwise, it prompts the user to select an alternative date or doctor slot.

To book an appointment, the system asks the patient to verify their personal profile details such as name, phone number, email address, age, gender, blood group, and address. Next, it processes payment verification via the integrated Razorpay API gateway, confirms the booking, and automatically updates the hospital appointment database and patient history ledger.

2. REQUIREMENTS

2.1 FUNCTIONAL REQUIREMENTS

2.1.1 User Account

The patient, who will henceforth be called the 'user', is presented with key options by the hospital management system during interaction. A user can interact with the system governed by whether they are a guest visitor, a registered patient, a doctor, or an administrator.

A user who has registered with the hospital previously will possess a unique User ID and password. A registered user can check doctor availability, book appointment slots, make payments, and view digital prescriptions by logging into the system. A new user would either have to: (a) Register with the system by providing personal information and profile details, or (b) Log into the system as a guest to perform general searches.

2.1.2 Registration and Creation of User Profile

The system requires a user to register in order to carry out any transactional operations except for checking doctor availability. It prompts the user for the following required details: User ID/Email, Password, First Name, Last Name, Gender, Date of Birth, Phone Number, Address, City, State, Blood Group, and Profile Avatar photo.

2.1.3 Quick Search & Doctor Discovery

Here we provide a Quick Search facility for any user to search doctor schedules and department listings without logging in. After logging in, the system requests the user to enter search criteria: Department Name, Doctor Name, or Specialization. The system queries the backend database ('DoctorMaster') to fetch available doctors, room numbers, consultation fees, and active status.

2.1.4 Making Appointments / Booking / Confirmation & Payment

After selecting a doctor and desired 30-minute time slot, the system validates slot availability to prevent double-booking. If confirmed, the system interacts with the Razorpay API gateway for fee processing. Upon successful payment verification, the system updates the reservation database DB-Appointment, decrements slot capacity, and emits confirmation receipts.

2.1.5 View Booking History & Medical Records

The system allows patients to view all past and upcoming consultation records. It accesses UserBookingTable and retrieves appointment ID, doctor details, schedule date, payment status, and digital prescriptions in a tabular format.

2.1.6 Administrator & Doctor Functionality

Administrators log in to manage doctors, add clinical departments, update pharmacy medicine stock, view patient rosters, and review transaction ledgers.

2.2 NON FUNCTIONAL REQUIREMENTS

- **Performance Capacity:** HMS is optimized to handle at least 1000 transactions/inquiries per second utilizing Spring Boot 3.x stateless REST and Hibernate ORM.
- **Constraint Boundary:** HMS shall process all database transactions within ACID compliance boundaries and respond to API queries under 200ms.
- **Hardware Interfaces:** Intel Core i3 Processor 2.4 GHz or equivalent, 4 GB RAM, 20 GB free storage.
- **Software Interfaces:** Windows 10/11 / Linux OS, MySQL 8.0 DB, Spring Boot 3.x JDK 17/21, React 18 SPA, Web Browsers.

3. DESIGN — DATABASE SCHEMA (10 CORE RELATIONAL TABLES)

Table 1: users

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	userid	BigInt	20	0 (No)
Unique	email	Varchar	255	0 (No)
None	password	Varchar	255	0 (No)
None	firstName	Varchar	30	1 (Yes)
None	lastName	Varchar	30	1 (Yes)
None	contactdetails	Varchar	10	0 (No)
None	address	Varchar	255	1 (Yes)
None	profile_photo	Varchar	255	1 (Yes)
None	date_of_birth	Date	-	0 (No)
None	timeofcreation	Timestamp	-	0 (No)
Enum	UserRole	Varchar	50	1 (Yes)

Table 2: patients

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	patient_id	BigInt	20	0 (No)
FK (Unique)	user_id	BigInt	20	0 (No)
None	blood_group	Varchar	10	1 (Yes)
None	emergency_contact_name	Varchar	50	1 (Yes)
None	emergency_contact_number	Varchar	15	1 (Yes)
None	emergency_contact_relation	Varchar	20	1 (Yes)
None	description	Varchar	255	1 (Yes)

Table 3: doctor

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	doctorid	BigInt	20	0 (No)
FK (Unique)	user_id	BigInt	20	0 (No)
Foreign Key	department_id	BigInt	20	1 (Yes)
None	specialization	Varchar	255	0 (No)
None	qualification	Varchar	255	0 (No)
None	yearsOfExperience	Int	11	0 (No)
None	consultationFee	Double	10,2	0 (No)
Unique	licenseNumber	Varchar	255	0 (No)
None	roomNumber	Int	11	1 (Yes)
Enum	availabilityStatus	Varchar	50	1 (Yes)
None	description	Varchar	1000	1 (Yes)

Table 4: department

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	departmentId	BigInt	20	0 (No)
Unique	departmentName	Varchar	255	0 (No)
None	description	Varchar	500	1 (Yes)

Table 5: appointments

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	appointment_id	BigInt	20	0 (No)
Foreign Key	patient_id	BigInt	20	0 (No)
Foreign Key	doctor_id	BigInt	20	0 (No)
Foreign Key	department_id	BigInt	20	0 (No)
None	appointment_date	Date	-	0 (No)
None	appointment_start_time	Time	-	0 (No)
None	appointment_end_time	Time	-	0 (No)
Enum	appointmentType	Varchar	50	0 (No)
Enum	status	Varchar	50	0 (No)
None	remarks	Varchar	500	1 (Yes)

Table 6: payments

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	payment_id	BigInt	20	0 (No)
Foreign Key	appointment_id	BigInt	20	0 (No)
Foreign Key	patient_id	BigInt	20	0 (No)
Foreign Key	doctor_id	BigInt	20	0 (No)
Unique	razorpay_order_id	Varchar	100	0 (No)
Unique	razorpay_payment_id	Varchar	255	1 (Yes)
None	razorpay_signature	Varchar	500	1 (Yes)
Unique	receipt_number	Varchar	255	0 (No)
None	amount	Decimal	10,2	0 (No)
None	currency	Varchar	10	0 (No)
Enum	payment_method	Varchar	50	1 (Yes)
Enum	order_status	Varchar	50	1 (Yes)
Enum	payment_status	Varchar	50	1 (Yes)
None	paid_at	Timestamp	-	1 (Yes)
None	created_at	Timestamp	-	1 (Yes)

Table 7: medicine_master

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	medicine_id	BigInt	20	0 (No)
None	medicine_name	Varchar	100	0 (No)
None	generic_name	Varchar	100	1 (Yes)

None	manufacturer	Varchar	100	1 (Yes)
None	strength	Varchar	50	1 (Yes)
None	dosage_form	Varchar	50	1 (Yes)
None	is_active	Boolean	1	0 (No)

Table 8: prescription

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	prescription_id	BigInt	20	0 (No)
FK (Unique)	appointment_id	BigInt	20	0 (No)
None	diagnosis	Varchar	500	0 (No)
None	notes	Varchar	1000	1 (Yes)
None	createdAt	Timestamp	-	1 (Yes)

Table 9: prescription_medicine

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	prescription_medicine_id	BigInt	20	0 (No)
Foreign Key	prescription_id	BigInt	20	0 (No)
Foreign Key	medicine_id	BigInt	20	0 (No)
None	dosage	Varchar	50	0 (No)
None	frequency	Varchar	50	0 (No)
None	duration	Varchar	50	0 (No)
None	instructions	Varchar	300	1 (Yes)
None	quantity	Varchar	30	1 (Yes)

Table 10: password_reset_token

Key Type / Constraint	Column Name	Data Type	Length	Allow Null
Primary Key	id	BigInt	20	0 (No)
Unique	token	Varchar	255	0 (No)
Foreign Key	user_id	BigInt	20	1 (Yes)
None	expiryTime	Timestamp	-	0 (No)
None	used	Boolean	1	0 (No)

4. CODING STANDARDS IMPLEMENTED

Identifier	Case	Examples	Additional Notes
Class	Pascal	Patient, DoctorService, AppointmentController	Class names are nouns. No '_' signs allowed.
Method	Camel	getPatientById, bookAppointment, processPayment	Methods use verbs describing actions.
Parameter	Camel	patientId, doctorDTO, appointmentDate	Descriptive parameter names indicating role.
Interface	Pascal with 'I'	PatientRepository, PaymentService	Interface definitions for JPA repositories.
Property	Pascal	ConsultationFee, AppointmentStatus	Noun phrases for entity properties.
Private Member	_camelCase	_patientService, _doctorRepo	Underscore camel casing for injected private fields.
Exception Class	Pascal with 'Exception'	ResourceNotFoundException, UnauthorizedAccessException	Suffix with 'Exception' for custom application exceptions.

5. TEST REPORT & FUNCTIONAL VERIFICATION

SR NO	TEST CASE	EXPECTED RESULT	ACTUAL RESULT	ERROR MESSAGE
1	Register Page	Redirected to login / patient dashboard	OK	Nothing
2	Login Page	Pop-up token issued, dashboard loaded	OK	Please enter valid credentials.
3	Reset Login	Password reset link sent to registered email	OK	Nothing
4	Quick Doctor Search	Returns matching doctors & time slots	OK	Nothing
5	Booking Ticket/Slot	Validates slot availability and saves draft	OK	Nothing
6	Check Login Status	Verifies active JWT token authority	OK	Nothing
7	Add Patient Details	Appends patient PHI to booking ledger	OK	Nothing
8	Goto Payment Gateway	Generates Razorpay order ID and modal	OK	Nothing
9	Confirm Payment	Verifies signature and updates DB to PAID	OK	Nothing
10	Cancel Transaction	Reverts slot status to AVAILABLE	OK	Nothing
11	View Prescriptions	Displays digital prescription and dosage	OK	Nothing
12	Logout	Clears JWT token and redirects to home	OK	Nothing

6. PROJECT MANAGEMENT STATISTICS & MILESTONES

DATE	WORK PERFORMED	SLC PHASE	ADDITIONAL NOTES
APR 12, 2026	Project Kickoff & Initial Requirements Gathering	Feasibility Study	Met hospital administration stakeholders to discuss problem statement, scope, and vision.
APR 25, 2026	SRS Document Draft & Validation	Requirement Analysis	Finalized functional requirements for Patient, Doctor, and Admin roles.

JUN 10, 2026	System Architecture & Relational DB Design Approval	Design Phase	Finalized ER Diagrams (10 tables), DFD Level 1, and Spring Boot component architecture.
JUL 02, 2026	Backend Foundation Setup (Spring Boot 3.x, MySQL DB)	Design & Setup	Configured Spring Security, BCrypt, database schemas, and JPA entities.
JUL 08, 2026	Development of Authentication & User APIs	Coding Phase	Created /api/auth endpoints for registration, JWT login, and profile management.
JUL 14, 2026	Patient & Doctor Portal REST Endpoints	Coding Phase	Developed doctor discovery, availability status, slot booking, and department services.
JUL 20, 2026	React 18 SPA Frontend Development	Coding Phase	Built responsive UI views for Patient Dashboard, Doctor Search, and Appointment Modals.
JUL 26, 2026	Admin Control Panel & Hospital Roster Management	Coding Phase	Implemented doctor registration, department assignment, and appointment oversight pages.
JUL 30, 2026	Razorpay Payment Gateway & Email Integration	Coding Phase	Integrated Razorpay API order creation, HMAC signature validation, and reset emails.
AUG 02, 2026	Pharmacy Inventory & Digital Prescription Module	Coding Phase	Developed medicine master stock control, dosage frequency, and prescription issuance.
AUG 05, 2026	Integration Testing & Security RBAC Audit	Testing Phase	Executed JUnit 5 integration tests and verified role permissions (PATIENT, DOCTOR, ADMIN).
AUG 07, 2026	User Acceptance Testing (UAT) with Hospital Staff	Testing Phase	Trial booking sessions conducted with hospital admin staff and medical officers.
AUG 09, 2026	Bug Fixes, UI Polish & Performance Tuning	Debugging Phase	Optimized SQL query indexing and resolved slot conflict validation edge cases.
AUG 11, 2026	Final Project Submission & Cloud Server Deployment	Deployment Phase	Deployed production build of Spring Boot backend and React SPA frontend.